Digital Signature를 이용한 무결성 기능을 지원하는

파일 전송 SW 구현

2026. 6. 1.

팀 해시브라운 

강정우
이원준
최재훈

INDEX

목차

목차 1 프로젝트 목표

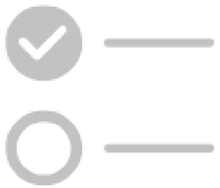
목차 2 계획 대비 진행상황

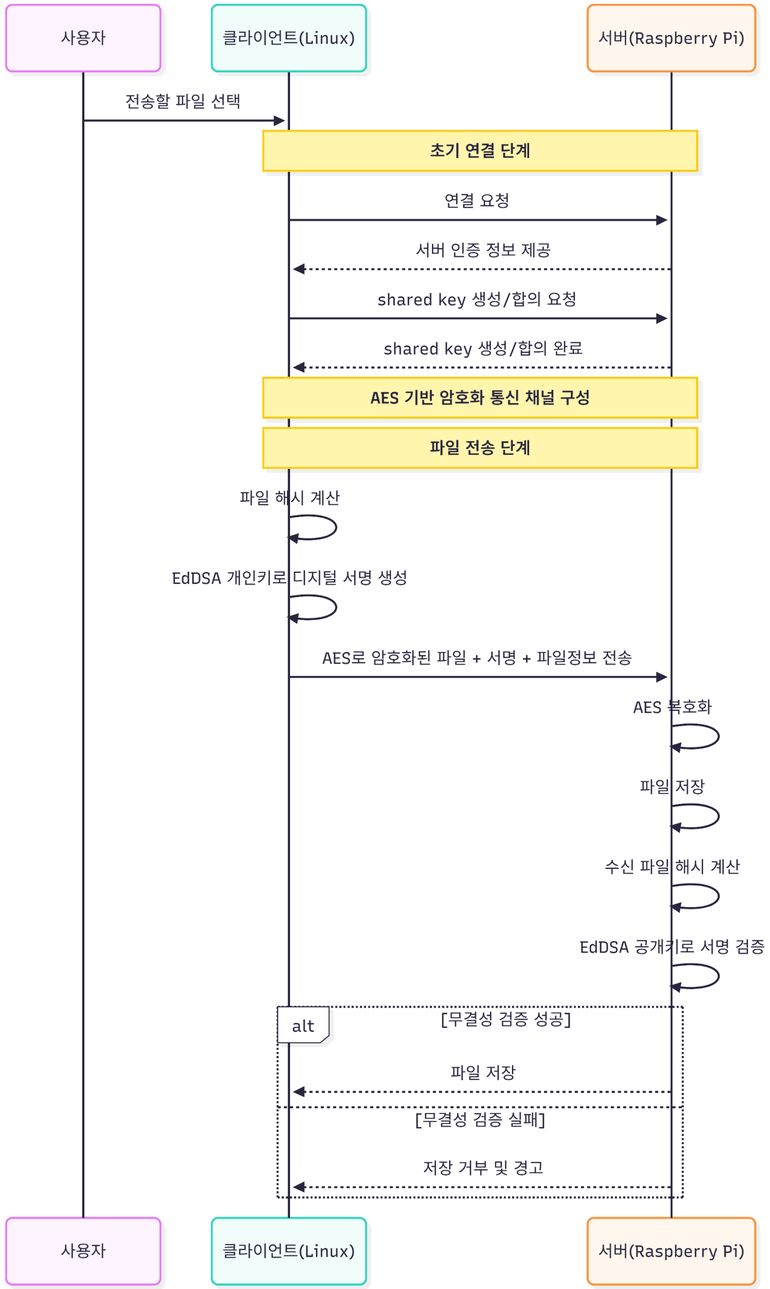
목차 3 역할 분담

목차 4 설계 사양

목차 5 연구 내용 및 중간 결과물

목차 6 프로젝트 향후 계획





연구 목적

EdDSA 기반 디지털 서명을 적용하여
전송 파일의 무결성을 보장하고

디지털 서명이 지원하지 않는 기밀성은
공유키 기반 AES 암호화 통신으로 보완하여

안전한 파일 전송 소프트웨어를 구현

연구 일정

연구내용	3월	4월	5월	6월	7월	8월	9월	10월
관련 내용 학습	■	■	■					
개발 환경 구축		■						
파일전송, 디지털 서명 모듈 구현		■	■	■				
AES 암호화 모듈 구현				■	■			
통합 및 기능 테스트					■	■	■	
최종 보고서 작성							■	
발표 자료 제작 및 시연 준비							■	■
최종 발표 및 산출물 제출								■

연구 일정

연구내용	3월	4월	5월	6월	7월	8월	9월	10월
관련 내용 학습	■	■	■					
개발 환경 구축		■						
파일전송, 디지털 서명 모듈 구현		■	■	■				
AES 암호화 모듈 구현				■	■			
통합 및 기능 테스트					■	■	■	
최종 보고서 작성							■	
발표 자료 제작 및 시연 준비							■	■
최종 발표 및 산출물 제출								■

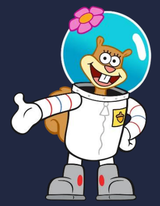
역할 분담

강정우



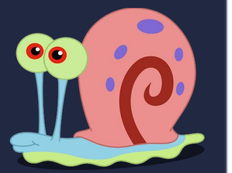
파일 전송 구현 및
OpenSSL 사용법 학습

이원준



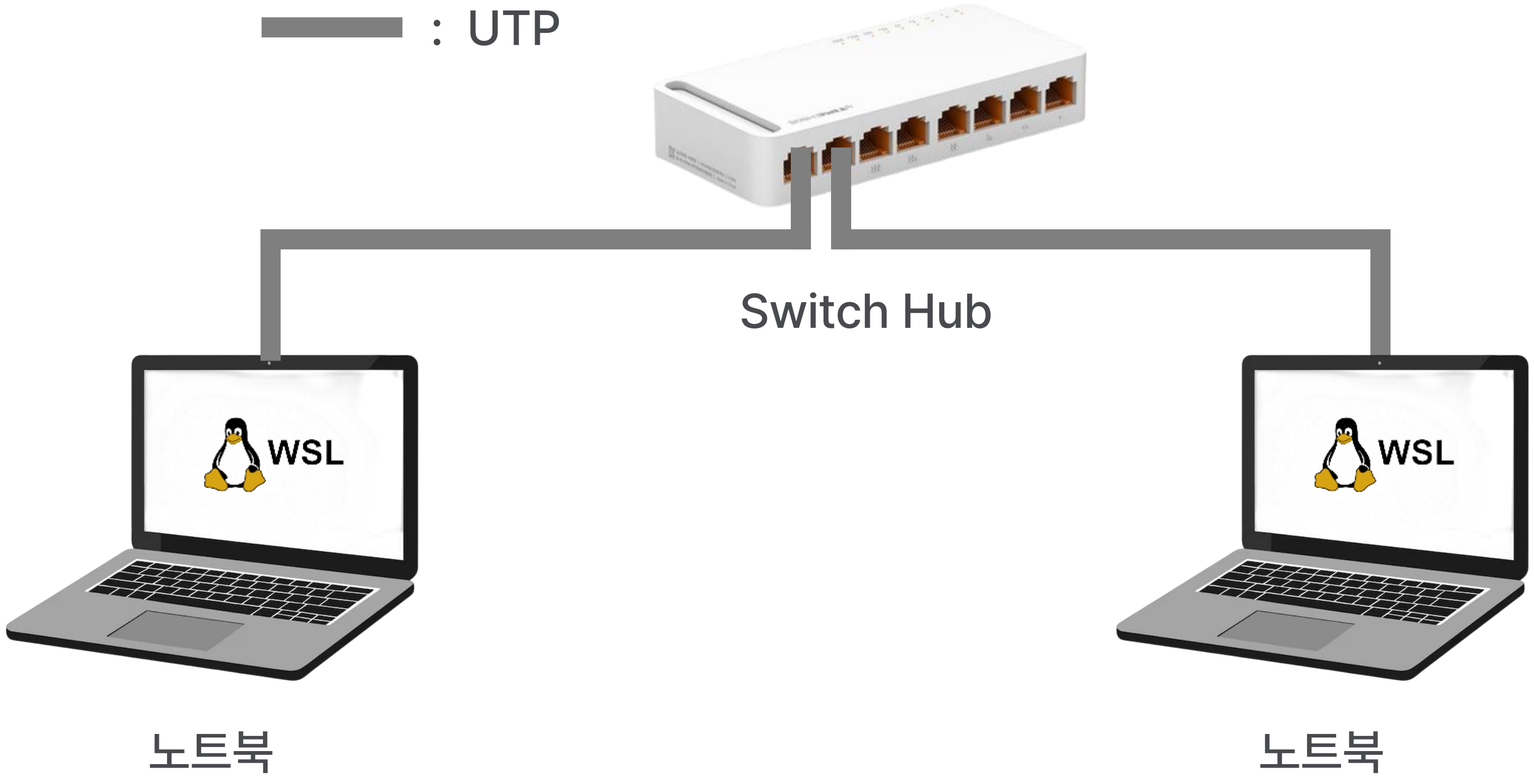
디지털 서명 및
무결성 검증 모듈 구현
(OpenSSL)

최재훈

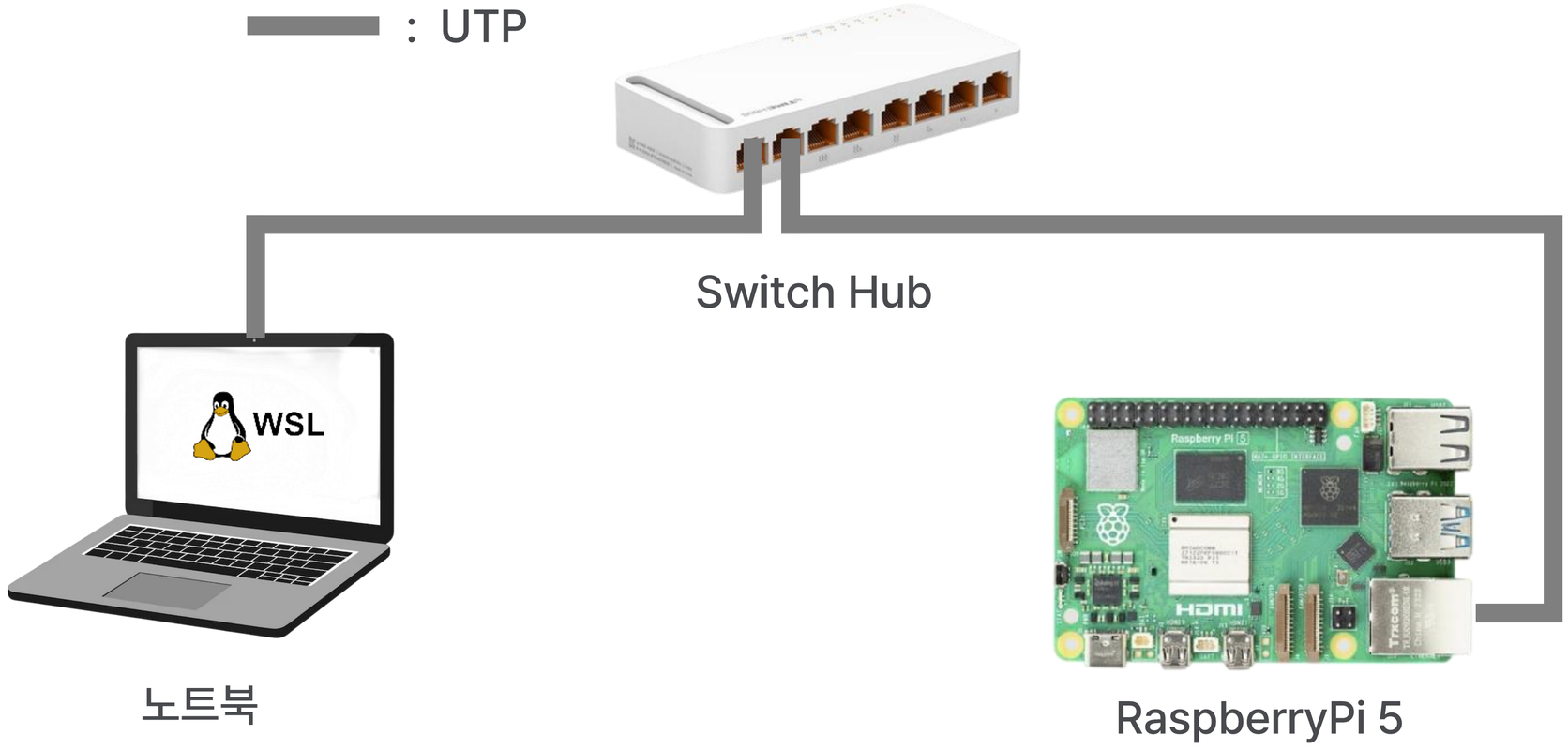


개발 환경 구성
기능 통합 및 테스트
보고서 작성

설계 사양(현재 테스트 환경)



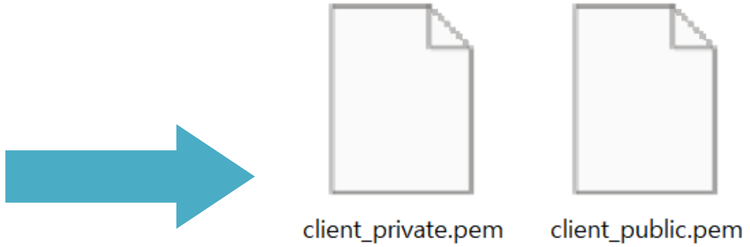
설계 사양(최종 적용 환경)



사전 준비 – 키 쌍 생성

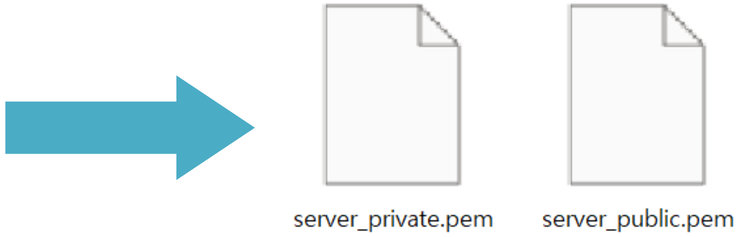
```
# 클라이언트 개인키 생성
openssl genpkey -algorithm ed25519 -out client_private.pem

# 클라이언트 공개키 추출
openssl pkey -in client_private.pem -pubout -out client_public.pem
```




```
# 서버 개인키 생성
openssl genpkey -algorithm ed25519 -out server_private.pem

# 서버 공개키 추출
openssl pkey -in server_private.pem -pubout -out server_public.pem
```



사전 준비 – 키 쌍 생성

클라이언트 측 보유

client_private.pem
(서명 생성 시 사용)

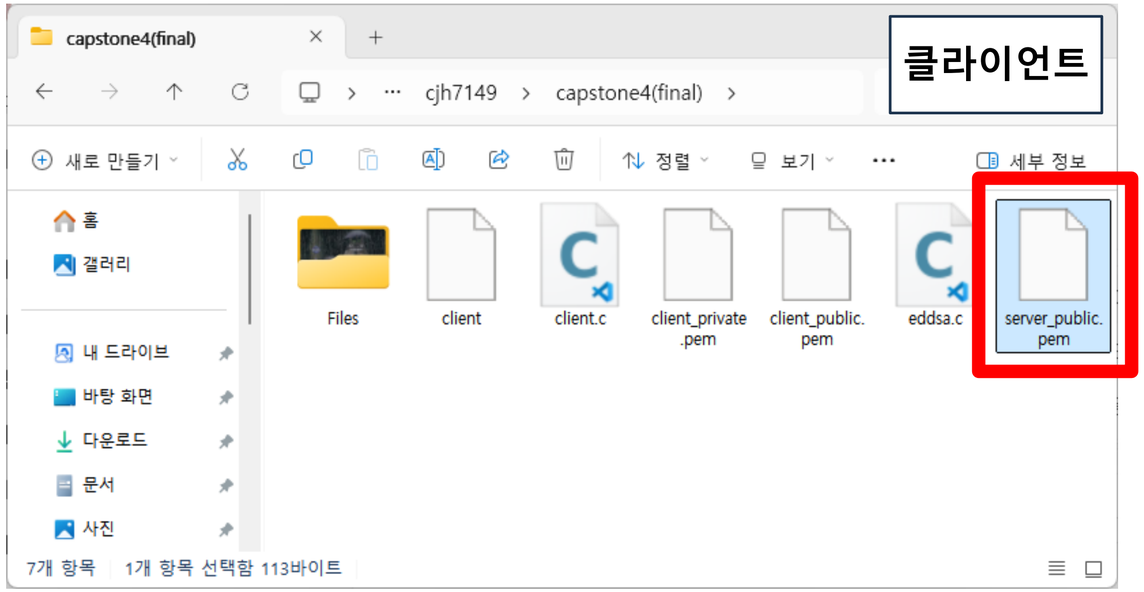
server_public.pem
(서명 검증 시 사용)

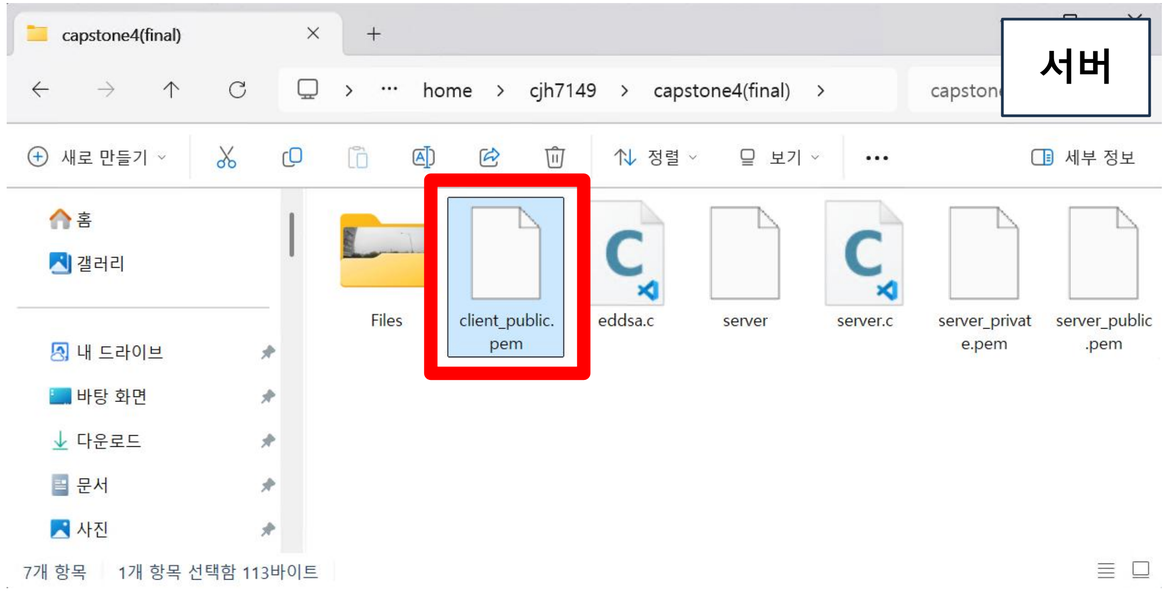
서버 측 보유

server_private.pem
(서명 생성 시 사용)

client_public.pem
(서명 검증 시 사용)

사전 준비 – 키 쌍 생성





각 주체는 자신의 개인키와 상대방의 공개키를 보관

서명 생성

client.c

```
make_signed_info(&signed_info,
    filename, filesize, hash);

eddsa_sign_with_key(
    (unsigned char *)&signed_info,
    sizeof(signed_info),
    &sign, &sign_len,
    CLIENT_PRIVATE_KEY
);
```

eddsa.c

```
// 1. 개인키 읽기
pkey = PEM_read_PrivateKey(fp, NULL, NULL, NULL);

// 2. 서명 컨텍스트 초기화 (NULL = EdDSA 내장)
EVP_DigestSignInit(ctx, NULL, NULL, NULL, pkey);

// 3. 서명 길이 확인 후 메모리 할당
EVP_DigestSign(ctx, NULL, &sig_len, msg, msg_len);
*sig = malloc(*sig_len);

// 4. 실제 서명값(64 byte) 생성
EVP_DigestSign(ctx, *sig, &sig_len, msg, msg_len);
```

1. Signed_Info 구조체 구성
파일명+크기+SHA-256

▶

2. 개인키 로드 (PEM)
client_private.pem

▶

3. EdDSA 컨텍스트 초기화
해시 알고리즘 NULL

▶

4. 서명값 생성 (64 byte)
EVP_DigestSign ×2

서명 검증

수신된 파일 데이터

▼

SHA-256 재계산
recv_hash 계산

▼

해시 비교 (hash_ok)
recv_hash == info.hash

▼

서명 검증 (sign_ok)
client_public.pem 사용

▼

파일 저장 성공
tmp → 정식 파일 rename

server.c

```c
// 1. 해시 비교
int hash_ok = ok && memcmp(recv_hash,
    info.hash, SHA256_DIGEST_LENGTH) == 0;

// 2. 공개키 서명 검증
int sign_ok = hash_ok &&
    verify_signature(safe_name, &info,
    sign, CLIENT_PUBLIC_KEY);

// 3. 검증 성공 시에만 저장
int save_ok = sign_ok && rename(tmp, save);
```

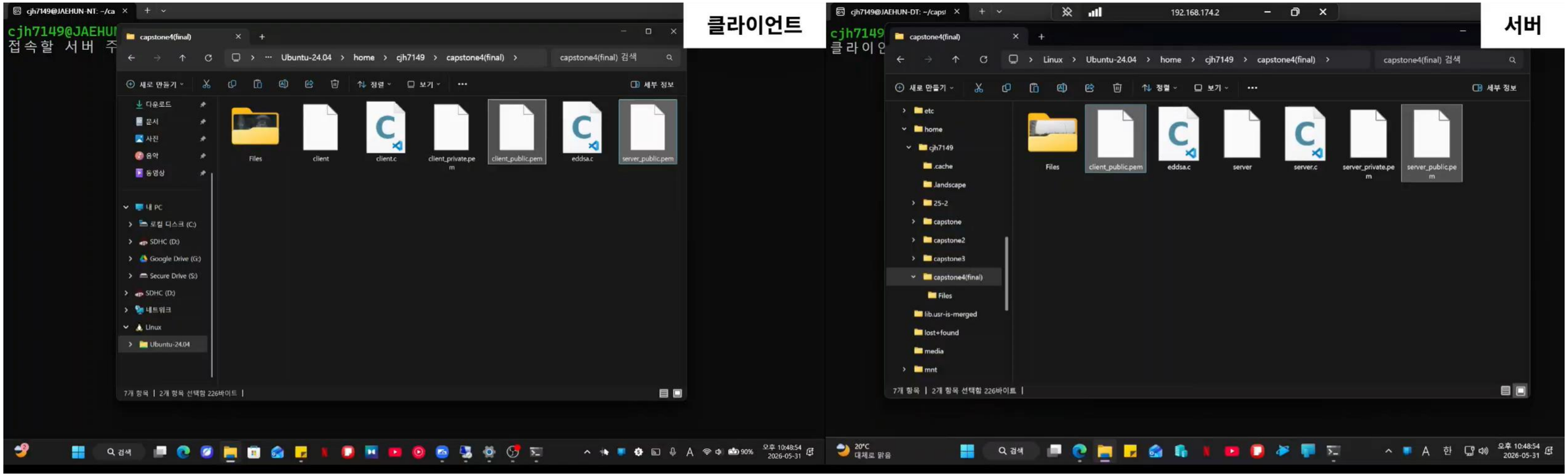
해시 재계산으로 위변조 탐지

수신 파일을 직접 SHA-256 → info.hash와 비교

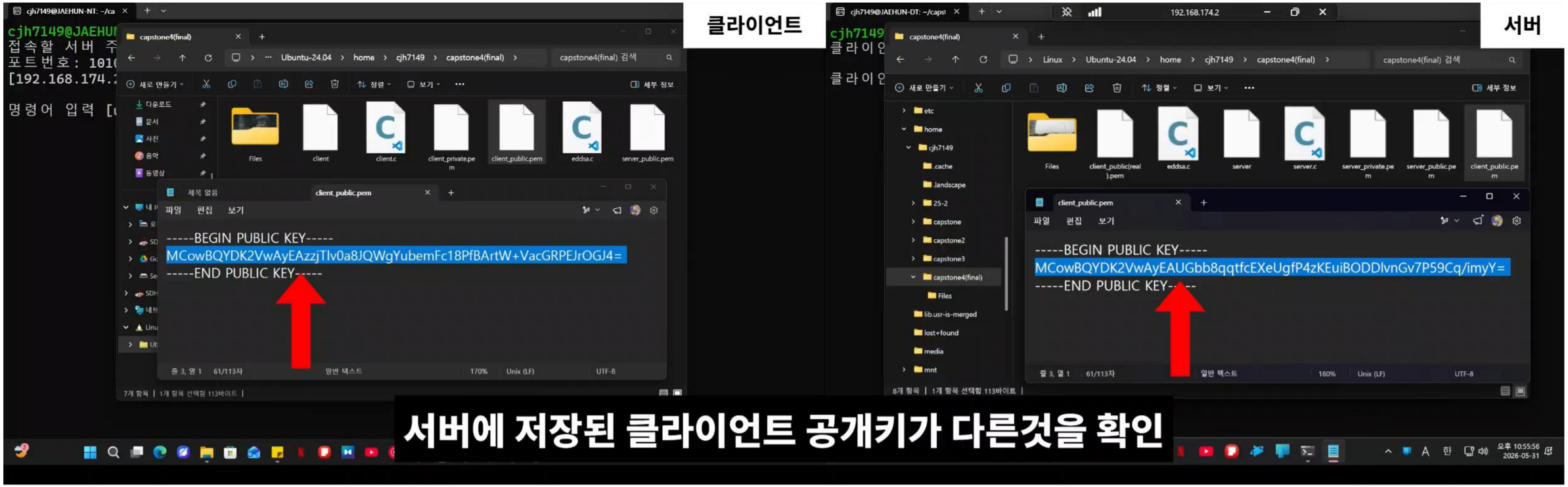
EdDSA 공개키 서명 검증

파일명+크기+해시 중 하나라도 바뀌면 검증 실패

시연 영상 – 정상 업로드 및 다운로드



시연 영상 – 공개키 변조


클라이언트
서버
-----BEGIN PUBLIC KEY-----
MCowBQYDK2VwAyEAzzjTlv0a8JQWgYubemFc18PfBArtW+VacGRPEJrOGJ4=
-----END PUBLIC KEY-----
-----BEGIN PUBLIC KEY-----
MCowBQYDK2VwAyEAUGbb8qqtfcEXeUgfP4zKEuiBODDlvnGv7P59Cq/imyY=
서버에 저장된 클라이언트 공개키가 다른것을 확인

연구 일정

연구내용	3월	4월	5월	6월	7월	8월	9월	10월
관련 내용 학습	■	■	■					
개발 환경 구축		■						
파일전송, 디지털 서명 모듈 구현		■	■	■				
AES 암호화 모듈 구현				■	■			
통합 및 기능 테스트					■	■	■	
최종 보고서 작성							■	
발표 자료 제작 및 시연 준비							■	■
최종 발표 및 산출물 제출								■

연구 일정

연구내용	3월	4월	5월	6월	7월	8월	9월	10월
관련 내용 학습	■	■	■					
개발 환경 구축		■						
파일전송, 디지털 서명 모듈 구현		■	■	■				
AES 암호화 모듈 구현				■	■			
통합 및 기능 테스트					■	■	■	
최종 보고서 작성							■	
발표 자료 제작 및 시연 준비							■	■
최종 발표 및 산출물 제출								■

프로젝트 향후 계획

AES 암호화 모듈 구현

파일 전송 + 디지털 서명 + AES 통합

라즈베리파이 환경에서 정상 작동 확인

감사합니다

팀 해시브라운 